

```
"""
Data Cleaning & Reporting Automation
Pipeline
-----
-----
Point this at any raw CSV/Excel
sales-style file. It will:
    1. Standardize column formats
(dates, text case, whitespace)
    2. Handle missing values (flag +
impute/drop based on a configurable
strategy)
    3. Remove exact and near-duplicate
records
    4. Flag statistical outliers
    5. Log every change made (for
auditability)
    6. Output a cleaned file + a
JSON/CSV change log

Usage:
    python clean_pipeline.py
input.csv output_cleaned.csv
"""

import pandas as pd
import numpy as np
```

```
import sys
import json
from datetime import datetime

LOG = {"steps": [], "summary": {}}

def log(step, detail):
    LOG["steps"].append({"step":
step, "detail": detail})
    print(f"[{step}] {detail}")

def load_data(path):
    if path.endswith(('.xlsx',
'.xls')):
        df = pd.read_excel(path)
    else:
        df = pd.read_csv(path)
    log("Load", f"Loaded {len(df)}
rows, {len(df.columns)} columns from
{path}")
    return df

def standardize_text(df):
    for col in
df.select_dtypes(include='object').co
lums:
```

```

        before_nulls =
df[col].isna().sum()
        df[col] =
df[col].astype(str).str.strip()
        df[col] =
df[col].replace({'nan': np.nan, '':
np.nan, 'None': np.nan})
        if col in ('Region',
'Category'):
            df[col] =
df[col].str.title()
            if col == 'Sales Rep':
                df[col] =
df[col].str.upper()
            log("Standardize text", "Trimmed
whitespace, normalized casing,
unified blank/NaN representations")
        return df

def standardize_dates(df, col='Order
Date'):
    if col not in df.columns:
        return df
    parsed = pd.to_datetime(df[col],
errors='coerce', format='mixed',
dayfirst=False)
    still_missing = parsed.isna() &
df[col].notna()

```

```

        if still_missing.any():
            parsed[still_missing] =
pd.to_datetime(df.loc[still_missing,
col], errors='coerce', dayfirst=True)
            df[col] = parsed
            log("Standardize dates", f"Parsed
'{col}' into consistent datetime
format; {parsed.isna().sum()}
unparseable values remain")
            return df

def handle_missing(df):
    missing_before = df.isna().sum()
    numeric_cols =
df.select_dtypes(include=np.number).c
olumns
    for col in numeric_cols:
        n_missing =
df[col].isna().sum()
        if n_missing > 0:
            median_val =
df[col].median()
            df[col] =
df[col].fillna(median_val)
            log("Fill missing
(numeric)", f"'{col}': filled
{n_missing} missing values with
median ({median_val:.2f})")

```

```

        categorical_cols =
df.select_dtypes(include='object').columns
    for col in categorical_cols:
        n_missing =
df[col].isna().sum()
        if n_missing > 0:
            df[col] =
df[col].fillna('Unknown')
            log("Fill missing
(categorical)", f"'{col}': filled
{n_missing} missing values with
'Unknown'")

    LOG["summary"]["missing_before"]
= {k: int(v) for k, v in
missing_before.items() if v > 0}
    return df

def remove_duplicates(df):
    before = len(df)
    exact_dupes =
df.duplicated().sum()
    df = df.drop_duplicates()

    key_cols = [c for c in
['OrderID'] if c in df.columns]

```

```

        near_dupes_removed = 0
    if key_cols:
        near_dupes_removed =
df.duplicated(subset=key_cols).sum()
        df =
df.drop_duplicates(subset=key_cols,
keep='first')

    log("Remove duplicates",
f"Removed {exact_dupes} exact
duplicate rows and
{near_dupes_removed} duplicate keys
({before} -> {len(df)} rows)")
    return df

def flag_outliers(df, col='Unit
Price'):
    if col not in df.columns:
        return df
    q1, q3 = df[col].quantile([0.25,
0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3
+ 1.5 * iqr
    df['Outlier_Flag'] = ((df[col] <
lower) | (df[col] > upper))
    n_outliers =
df['Outlier_Flag'].sum()

```

```
        log("Flag outliers", f"Flagged  
{n_outliers} rows in '{col}' outside  
[{lower:.2f}, {upper:.2f}]")  
    return df
```

```
def recompute_derived(df):  
    if 'Quantity' in df.columns and  
    'Unit Price' in df.columns:  
        df['Revenue'] =  
(df['Quantity'] * df['Unit  
Price']).round(2)  
        log("Recompute",  
"Recalculated 'Revenue' = Quantity x  
Unit Price after cleaning")  
    return df
```

```
def run_pipeline(input_path,  
output_path):  
    df = load_data(input_path)  
    df = standardize_text(df)  
    df = standardize_dates(df)  
    df = handle_missing(df)  
    df = remove_duplicates(df)  
    df = recompute_derived(df)  
    df = flag_outliers(df)  
  
    df.to_csv(output_path,
```

```

index=False)
    log("Save", f"Saved cleaned
dataset ({len(df)} rows) to
{output_path}")

    LOG["summary"]["rows_in"] = None
    LOG["summary"]["rows_out"] =
len(df)
    LOG["summary"]["run_at"] =
datetime.now().isoformat()

    with
open(output_path.replace('.csv',
'_log.json'), 'w') as f:
        json.dump(LOG, f, indent=2,
default=str)

    return df

if __name__ == '__main__':
    input_path = sys.argv[1] if
len(sys.argv) > 1 else
'raw_sales_data.csv'
    output_path = sys.argv[2] if
len(sys.argv) > 2 else
'cleaned_sales_data.csv'
    run_pipeline(input_path,
output_path)

```

